Pre-Lab Activity 1: Understanding useEffect Execution

```
import React, { useEffect } from "react";
function WelcomeComponent() {
  useEffect(() => {
    console.log("Component Loaded Successfully!");
  }, []); // Empty dependency array

  return <h2>Hello, Welcome to React useEffect Demo!</h2>;
}
export default WelcomeComponent;
```

Output:

Pre-Lab Activity 2: Dependency Array Behavior

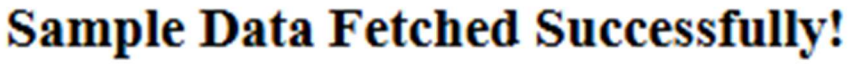
```
import React, { useState, useEffect } from "react";
function CounterComponent() {
  const [count, setCount] = useState(0);
  // Dependency with count
  useEffect(() => {
    console.log("Counter Updated");
  }, [count]);
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
export default CounterComponent;
```

Output:

Pre-Lab Activity 3: Simulating API Call using setTimeout

```
import React, { useState, useEffect } from "react";
function FakeApiComponent() {
  const [data, setData] = useState(null);
  useEffect(() => {
    setTimeout(() => {
      setData("Sample Data Fetched Successfully!");
    }, 2000);
  }, []);
  return (
    <div>
      {data ? <h3>{data}</h3> : <h3>Loading...</h3>}
    </div>
  );
}
export default FakeApiComponent;
```

Output:



Pre-Lab Activity 4: Basic Fetch API (Without useEffect)

```
import React from "react";
function FetchOnClick() {
  const fetchData = () => {
fetch("https://jsonplaceholder.typicode.com/users")
```

```
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.error("Error:", error));
 };
 return (
  <div>
   <button onClick={fetchData}>Fetch Data</button>
  </div>
 );
}
export default FetchOnClick;
```

Output:

Pre-Lab Activity 5: Async/Await Practice

```
import React from "react";
function AsyncAwaitExample() {
 const fetchData = async () => {
  try {
   const response = await fetch("https://jsonplaceholder.typicode.com/users");
   const data = await response.json();
   console.log(data);
  } catch (error) {
   console.error("Error fetching data:", error);
  }
 };
 return (
  <div>
   <button onClick={fetchData}>Fetch Data</button>
  </div>
 );
}
export default AsyncAwaitExample;
```

Output:

Pre-Lab Activity 6: Rendering Array Data Dynamically

```
import React, { useState } from "react";
function UserList() {
  const [users] = useState([
    { id: 1, name: "Rahul Sharma", email: "rahul@gmail.com" },
    { id: 2, name: "Priya Mehta", email: "priya@gmail.com" },
    { id: 3, name: "Amit Verma", email: "amit@gmail.com" }
  ]);
return (
    <div>
      <h2>User List</h2>
      <ul>
        {users.map(user => (
          <li key={user.id}>
            <strong>{user.name}</strong> - {user.email}
          </li>
        ))}
      </ul>
    </div>
  );
}
export default UserList;
```

Output:

Pre-Lab Record Questions:

1.When does useEffect execute?

useEffect executes after the component renders (after the DOM updates).

- No dependency array → Runs after every render.
- Empty dependency array [] → Runs only once (when component mounts).
- With dependencies [value] → Runs whenever the specified dependency changes.
- It can also return a cleanup function, which runs before the component unmounts or before the next effect execution.

---

2. What is the difference between .then() and async/await?

.then()	async/await
Uses promise chaining	Uses synchronous-like syntax
Can become complex with multiple chains	Cleaner and easier to read
Errors handled using .catch()	Errors handled using try-catch
Less readable for complex logic	More readable and structured

---

3. Why is loading state important during API calls?

Loading state is important because:

- It improves user experience.
- It informs users that data is being fetched.
- It prevents showing empty or broken UI.
- It avoids confusion during network delay.

---

4. Why must fetched data be stored in state?

Fetched data must be stored in state because:

- React re-renders the component when state changes.
- If data is stored in a normal variable, React will not detect the change.
- Storing data in state ensures the UI updates automatically when new data arrives.